

System Architecture Specification

Omni-Vector Cognitive Audio Pipeline — Legion-Jacked-Pipeline v1.0

Prepared By	Adam McCoy, AI Pipeline Engineer	Version	1.0.0
Project Type	Sovereign Edge Cognitive Audio System	Date	June 2026
Classification	Confidential — Client Handoff	Status	Final Draft

1.PROJECT OVERVIEW

This document specifies the architecture of the Omni-Vector Cognitive Audio Pipeline — a local-first, sovereign-edge system that fingerprints audio tracks into 37-dimensional mathematical vectors and stores them in LanceDB for millisecond-scale similarity search. The system bridges the gap between human language and hard DSP physics by feeding structured Pydantic math payloads to a locally-hosted Gemma 2b LLM via JINJA2 templates, enabling library-scale cognitive audio reasoning with zero data egress, zero API cost, and full forensic auditability. This is the foundational architecture for the Legion-Jacked-Pipeline — an autonomous agentic production system.

Design Principle

Zero data egress. Zero API cost. All DSP analysis, Pydantic validation, vector storage,

LLM inference, and audit logging are performed entirely on the local machine. No audio data, math payloads, or generated reasoning is transmitted to any third-party service at any stage of the pipeline.

2.SYSTEM COMPONENTS

The table below enumerates each logical component of the Legion-Jacked-Pipeline, the underlying technology and version configuration, and its designated role within the system.

Component	Technology	Version / Config	Role
DSP Analysis Layer	Python + librosa	librosa 0.10+, C++ backend	Computes RMS, Crest Factor, sub-bass energy (20-60 Hz), spectral centroid, harmonic chroma, LUFS, transient density, and stereo width correlation.
Pydantic DeltaEngine	Pydantic v2 + Python	Custom <code>DeltaEngine(target, live)</code> class	Subtracts live track math from target vector. <code>model_dump_json()</code> enforces rigid JSON — corrupt data crashes at ingestion, not silently downstream.
JINJA2 Prompt Engine	Jinja2	Dynamic template rendering	Injects <code>math_summary</code> JSON payload into structured LLM prompts. Includes Rosetta Stone system prompt with hardcoded DSP interpretation rules.
LLM Inference (Sovereign Edge)	Ollama + Gemma 2b	<code>gemma:2b</code> local instance	Reads Pydantic math payloads and reasons about acoustic physics. 16.08 tokens/sec on CPU. Zero data

Component	Technology	Version / Config	Role
			egress.
Vector Database	LanceDB	Local persistent mode	Stores 37-dim Omni-Vector fingerprints. Sub-20 ms similarity search at 50k+ track scale.
Blueprint Engine	Custom Python	v1.0	Maps song narrative structure: breakdown_position_sec, buildup_duration_sec, drop_intensity_delta — understands structural tension and release.
Orchestration Layer	Legion-Jacked-Pipeline	Python 3.11+	End-to-end pipeline: ingest → chunk → DSP → Pydantic → LanceDB → JINJA → Gemma → audit.
Audit Logger	Custom JSONL writer	Append-only	Logs 100% of query and inference events. Tamper-evident. Zero missed events in evaluation.
Future: MCP Server	Model Context Protocol	Planned	Will expose Ableton Live as an execution environment for the agentic producer.

3.DATA FLOW (STEP BY STEP)

The pipeline operates across two phases: an **ingestion phase** (steps 1–4) that fingerprints and indexes audio tracks, and a **query & inference phase** (steps 5–7) that handles real-time retrieval, LLM reasoning, and forensic audit logging.

1 Audio source (.wav / .mp3 / .aiff) → DSP Analysis Layer → 37-dimension raw physics dict (RMS, crest factor, sub_bass_energy, spectral centroid, chroma, LUFS, transient density, stereo width correlation, etc.)

- 2 Raw physics dict → Pydantic DeltaEngine — `DeltaEngine(target_vector, live_vector)`
→ `model_dump_json()` → sanitized `math_summary` JSON (crash-on-corrupt guarantee)
- 3 Blueprint Engine → maps `breakdown_position_sec`, `buildup_duration_sec`,
`drop_intensity_delta` → appended to `math_summary`
- 4 `math_summary` → LanceDB upsert → 37-dim Omni-Vector stored with track metadata
- 5 Query: new track DSP → DeltaEngine → `math_summary` → JINJA2 template → structured LLM prompt (with Rosetta Stone system prompt)
- 6 Prompt → Ollama (local) → Gemma 2b inference → acoustic physics reasoning output (EQ deltas, compression parameters, structural recommendations)
- 7 Every step 5-6 event → Audit Logger → JSONL entry appended (append-only, tamper-evident)

Crash-on-Corrupt Guarantee

The Pydantic `DeltaEngine` class enforces rigid schema validation at ingestion time. Malformed or missing DSP fields cause an immediate hard crash rather than propagating silently to the LLM inference layer. This ensures that every `math_summary` payload reaching Gemma is structurally valid and physically meaningful.

4. THE ROSETTA STONE (SYSTEM PROMPT ARCHITECTURE)

The system prompt is the critical calibration layer that prevents Gemma from misinterpreting DSP physics. LLMs naturally assume “Negative = Bad / Quiet.” The Rosetta Stone explicitly hardcodes DSP rules — for example: *“A NEGATIVE gain means the live track is too loud and the frequency is being CUT.”* This allows Gemma to pivot from calculating to reasoning — understanding *why* DSP deltas affect the energy flow and physical feel of the track, not merely what the numbers are.

Why This Matters

Without the Rosetta Stone, a raw DSP delta of -6.2 dB at 60 Hz would be interpreted as “the bass is quiet.” With the Rosetta Stone, Gemma understands: “the live sub-bass is 6.2 dB too loud relative to the target master — apply a shelf cut.” The system prompt is what turns a math-reading LLM into a cognitive audio engineer.

5.OMNI-VECTOR SCHEMA OVERVIEW

Each audio track is fingerprinted into a 37-dimensional Omni-Vector. The table below describes the major dimension groups, their key variables, and what the LLM learns to reason about from each group.

Dimension Group	Key Variables	What the LLM Learns
Dynamics	RMS, Crest Factor, LUFS Integrated	Compression depth, loudness headroom, transient preservation under limiting.
Sub-Bass Physics	sub_bass_energy (20-60 Hz)	“Heavy Bass” maps to <code>sub_bass_energy > 25000</code> and <code>crest_factor < 3.0</code> .
Spectral Profile	spectral_centroid, spectral_rolloff	Whether a track sits bright or dark. Centroid ≈ 2061 Hz = intimate club feel.
Harmonic Structure	harmonic_chroma_key, chroma_vector	Key, mode, and harmonic tension distributed across the full track duration.
Stereo Field	stereo_width_correlation	Phase cancellation risk and mono-compatibility of sub-bass content.
Transient Behavior	transient_density_hz	Drum sharpness under compression. Value of 4.5 = sharp rise times maintained.
Narrative	breakdown_position_sec, buildup_duration_sec, drop_intensity_delta	Structural tension and release — where the breakdown lands, how long the buildup runs, how hard the drop hits.

6.DEPLOYMENT SPECIFICATION

The Legion-Jacked-Pipeline runs entirely on a local machine with no containerization requirement for core operation. LLM inference is served by Ollama; the vector store persists to disk via LanceDB.

Parameter	Value
Runtime	Python 3.11+, local machine (Linux / macOS / Windows)
LLM Backend	Ollama (local) — install: <code>curl https://ollama.ai/install.sh sh</code>
Model	Gemma 2b — pull: <code>ollama pull gemma:2b</code>
Vector DB	LanceDB — install: <code>pip install lancedb</code>
GPU	Optional (CPU fully supported). GPU reduces P99 inference from ~4.7 s to ~1.5 s.
Inference Throughput (CPU)	16.08 tokens/sec (measured, Gemma 2b)
Data Egress	0 bytes — fully sovereign edge
API Cost	\$0.00 per query

7.NOTEBOOK REFERENCE

This pipeline was developed and validated in a Jupyter notebook environment. The notebook walks through the complete implementation sequence:

- DSP feature extraction with `librosa`
- Pydantic `DeltaEngine` class construction and validation
- JINJA2 template rendering with `math_summary` injection
- LanceDB upsert and similarity search
- Ollama / Gemma 2b inference with Rosetta Stone system prompt
- Blueprint Engine narrative mapping
- Audit log implementation and append-only verification

GitHub Notebook

[GITHUB_NOTEBOOK_URL] — placeholder to be replaced with actual repository URL prior to client delivery.

8.ROADMAP

The table below outlines the planned development phases for the Legion-Jacked-Pipeline beyond the current v1.0 release.

Phase	Feature	Status
v1.0	37-dim Omni-Vector fingerprinting + LanceDB + Gemma 2b reasoning	✓ Complete
v1.1	Swarm ingestion (automated Beatport Top 100 weekly hydration)	Planned
v2.0	MCP Server for Ableton Live — real-time agentic DAW integration	Planned
v2.1	Acoustic Meta-Trend Prediction (genre physics drift detection)	Planned
v3.0	Mathematical Reference Matching — auto-generate EQ/compression params to match any target master	Planned

Confidential — Prepared for [Client Name] | Adam McCoy, AI Pipeline Engineer |
Legion-Jacked-Pipeline v1.0 | June 2026

Notebook: [GITHUB_NOTEBOOK_URL] | This document is a proprietary deliverable.
Unauthorized reproduction or distribution is prohibited.